

Device Token Abuse Controls

Design Specification

Date: 2026-07-06

Status: Draft

Sources: Defense panel feedback, design grill with grill-with-docs skill, Oracle + Reviewer subagent consultations

Glossary: See CONTEXT.md (Device Token, Device Blocklist, Shadow Throttle, Registered Device Enrollment)

1. Problem

The current IP-based blocking mechanism (`ip:block:{ip}` in Redis, checked by `blocked_ip_middleware`) has a fundamental flaw: ISPs in the Philippines use **CGNAT and shared/dynamic IPs**. Blocking a single IP can block an **entire building or neighborhood**, including legitimate users reporting fires.

The defense panel identified this as a critical usability problem during the civilian contributor review. Additionally, **insider threats** from BFP personnel cannot be effectively managed at the IP level.

2. Design Overview

Replace IP-as-sole-identifier with a **server-issued signed device token** stored as a secure HTTP-only cookie. The token survives IP changes and provides per-browser-installation identity. IP-based blocking is retained as a fallback for uncooked clients (bots, curl, scripts).

Three pillars:

- **A) Device token** — server-issued, HMAC-SHA256 signed, versioned keyring, HttpOnly cookie
 - **B) Device blocklist** — separate `wims.device_blocklist` table + Redis hot-path, independent from IP blocklist
 - **C) Soft enforcement for public endpoints** — shadow throttle / CAPTCHA escalation instead of hard 403
-

3. Token Issuance and Storage

3.1 Token format

```
v1.<base64url_random_32_bytes>.<base64url_hmac_sha256>
```

- **v1** — key version prefix (enables rotation)
- Random 32 bytes (256 bits) from `os.urandom()`
- HMAC-SHA256 signed with `DEVICE_TOKEN_SIGNING_KEY` (or `DEVICE_TOKEN_SIGNING_KEY_V{n}` for rotated versions)

3.2 Bootstrap flow

1. **First request** — user visits any page, no `wims_device_token` cookie present
1. 2. **Token generation** — backend generates token, signs with active key version, sets as secure cookie:
 - Name: `wims_device_token`
 - Flags: `HttpOnly, Secure, SameSite=Lax`
 - Path: `/`
 - Max-Age: 1 year
1. 3. **Hash exposure** — backend computes `device_token_hash = SHA-256(raw_token)` and exposes it via `GET /api/device/token` bootstrap endpoint returning `{ device_token_hash: "abc123..." }`
1. 4. **Frontend storage** — frontend stores **only the hash** in IndexedDB/localStorage for offline/SW continuity. The raw signed token never reaches JavaScript.

3.3 Key rotation

Follows the existing `WIMS_MASTER_KEY` versioned keyring pattern:

Env var	Purpose
<code>`DEVICE_TOKEN_SIGNING_KEY`</code>	Initial signing key (version 1)
<code>`DEVICE_TOKEN_SIGNING_KEY_V2`</code>	Rotated signing key (version 2, optional)
<code>`DEVICE_TOKEN_SIGNING_KEY_ACTIVE_VERSION`</code>	Which version to use for new tokens (default: <code>`1`</code>)

Verification: parse version prefix from token → look up matching key → verify HMAC → hash body for Redis keying. When an old-version token is presented, re-issue with the active version.

4. Middleware Ordering

All requests (exempting `/health`, `/metrics`):

```
1. correlation_id_middleware → request.state.correlation_id
2. device_token_middleware (NEW) → read cookie → verify → inject hash
3. device_block_middleware (NEW) → check device:block:{hash} →
public: inject flag for downstream throttle
auth'd: 403
4. blocked_ip_middleware (ex) → check ip:block:{ip} → 403
5. rate_limit_middleware → rate limit (ip + device hash)
```

All middlewares are **fail-open** — if Redis is down or HMAC verification fails, the request proceeds with a warning logged.

4.1 device_token_middleware

- Cookie present and valid → verify HMAC → inject `request.state.device_token_hash`
- Cookie absent → generate new token → sign → set cookie → inject hash

- Cookie corrupted (HMAC mismatch) → treat as absent → issue new token. Logs a warning at most once per minute per IP.

4.2 device_block_middleware

- Extract `device_token_hash` from `request.state`
- If hash present → Redis `EXISTS device:block:{hash}` → branch by audience:
- **Public endpoints** (`/api/civilian/`, `/tracking`, `/api/v1/public/`): inject `request.state.device_blocked = True` for downstream CAPTCHA/throttle. No 403.
- **Authenticated endpoints** (admin, validator, analyst, regional): **403** `{"detail": "Device blocked"}`
- If no hash (headless bot) → fall through to `blocked_ip_middleware`

5. Blocklist Storage

5.1 Separate `wims.device_blocklist`` table

```
CREATE TABLE IF NOT EXISTS wims.device_blocklist (
  block_id SERIAL PRIMARY KEY,
  device_token_hash TEXT NOT NULL, -- SHA-256 of the signed token
  blocked_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  expires_at TIMESTAMPTZ, -- NULL = permanent
  is_permanent BOOLEAN NOT NULL DEFAULT false,
  blocked_by UUID, -- admin user_id
  block_reason TEXT,
  threat_log_id INTEGER, -- which alert triggered it
  user_agent TEXT, -- captured at block time
  authenticated_user_id UUID, -- from JWT, if logged in
  is_active BOOLEAN NOT NULL DEFAULT true
);

CREATE INDEX IF NOT EXISTS idx_device_blocklist_hash ON
wims.device_blocklist(device_token_hash);
CREATE INDEX IF NOT EXISTS idx_device_blocklist_active ON wims.device_blocklist(is_active)
WHERE is_active = true;

ALTER TABLE wims.device_blocklist ENABLE ROW LEVEL SECURITY;

DROP POLICY IF EXISTS device_blocklist_admin_all ON wims.device_blocklist;
CREATE POLICY device_blocklist_admin_all ON wims.device_blocklist
FOR ALL
USING (wims.current_user_role() IN ('SYSTEM_ADMIN'))
WITH CHECK (wims.current_user_role() IN ('SYSTEM_ADMIN'));
```

5.2 Redis keys

Key	Value	TTL	Purpose
<code>`device:block:{hash}`</code>	<code>`"1"`</code>	Block duration	Hot-path lookup
<code>`device:block:{hash}:meta`</code>	<code>`{reason, blocked_at, admin_id, user_agent}`</code>	Same	Metadata for admin panel

5.3 ``wims.ip_blocklist`` stays IP-only

The existing `ip_blocklist` table is **not modified**. No `device_token_hash` column. Device blocking and IP blocking are independent systems — the admin chooses which to use from the security log view.

6. Security Log Enrichment

6.1 Problem

Suricata alert ingestion (`suricata_ingestion.py`) reads EVE JSON files and inserts rows into `wims.security_threat_logs`. It has no access to device tokens because tokens come from the web request middleware, not from Suricata's packet-level detection.

6.2 Solution: Redis telemetry correlation

1. **Writer (middleware):** On every web request, the `device_token_middleware` writes a lightweight telemetry record to Redis:
 - Key: `device:telemetry:{normalized_source_ip}`
 - Value: `{ device_token_hash, user_agent, authenticated_user_id, timestamp, path }`
 - TTL: **120 seconds**
2. **Reader (Suricata ingestion):** Before inserting a new security log row, the ingestion pipeline:
 - Reads `device:telemetry:{src_ip}` from Redis
 - If a single unique device token hash is found → attaches it to the security log row
 - If multiple hashes for the same IP within the TTL window → leaves `device_token_hash` NULL (ambiguous)
 - Stores correlation metadata so admins know it's inferred:
 - `device_correlation_source: "redis_telemetry"`
 - `device_correlation_confidence: "high" | "ambiguous"`
 - `device_observed_at: timestamp from telemetry`
3. **New columns on `wims.security_threat_logs`:**
 - `device_token_hash` TEXT — NULL for uncooked clients or ambiguous matches
 - `device_correlation_source` TEXT — `"redis_telemetry"`, or NULL
 - `device_correlation_confidence` TEXT — `"high"`, `"ambiguous"`, or NULL
 - `device_observed_at` TIMESTAMPTZ — when the device was last seen for this IP

7. Enforcement Tiers

Endpoint category	Device blocked?	Response
Public <code>/api/civilian/</code>	Yes	**Soft** — inject <code>request.state.device_blocked`</code> . Shared <code>check_device_abuse()</code> dependency tightens rate limit, requires CAPTCHA, or quarantines to validator. No hard 403.
Public <code>/tracking`</code>	Yes	**Soft** — same as above. Tracking page still loads but throttled.

Endpoint category	Device blocked?	Response
Public <code>/api/v1/public/</code>	Yes	Soft — same escalation.
Admin <code>/api/admin/</code>	Yes	Hard 403 — <code>{"detail": "Device blocked"}</code>
Validator dashboard	Yes	Hard 403
Regional dashboard	Yes	Hard 403
Analyst dashboard	Yes	Hard 403
No cookie (bot)	N/A	Fall through to IP blocklist / Suricata / nginx bot-blocker

7.1 Shared abuse-check dependency

A FastAPI dependency `check_device_abuse()` that civilian/public routes add to their endpoint:

```
async def check_device_abuse(request: Request):
    if getattr(request.state, "device_blocked", False):
        # Extend existing civilian:backoff pattern
        token_hash = request.state.device_token_hash
        if token_hash:
            await r.setex(f"civilian:backoff:device:{token_hash}", 3600, "1")
```

The existing `civilian:backoff:{ip_hash}` pattern already exists in the public abuse controls — this adds the device token key to the same check.

8. Admin UI

8.1 BlockedIpsPanel — two tabs

[Blocked IPs] [Blocked Devices]

Blocked IPs tab: Existing view — unchanged. Shows `source_ip`, block episodes, expiry, Unblock button.

Blocked Devices tab: Same layout style, keyed by `device_token_hash`:

- Truncated hash (first 12 chars): `abc123def456...`
- User-agent (truncated to 60 chars)
- Block count and reason
- Linked username if `authenticated_user_id` was captured
- Expiry / permanent badge
- Unblock button: "Unblock device abc123..."

8.2 Blocking flow (admin monitoring page)

When an admin clicks "Block source IP" on a security log row:

1. If `device_token_hash` is present on the row → show a choice:
 - **"Block this device"** — blocks via `device_blocklist`
 - **"Block IP"** — blocks via existing `ip_blocklist`
1. 2. If no `device_token_hash` → fall back to current IP-only blocking

1. 3. **Bulk action:** When 2+ security logs are selected:
 - If all selected logs share the same `device_token_hash` → block device
 - If multiple hashes → show grouping preview ("3 logs for device A, 2 for device B") requiring explicit choice
 - Never silently fall back to IP blocking from mixed hashes

8.3 API endpoints

Method	Path	Purpose	
	<code>/api/admin/device-blocklist`</code>	List blocked devices	
	<code>/api/admin/device-blocklist/{token_hash}`</code>	Soft-unblock a device	
	<code>/api/admin/security-log/{log_id}/block`</code>	Block device or IP from a security log row (body: <code>{ type: "device"</code>	"ip" })

Pattern follows existing `/api/admin/ip-blocklist` resource naming.

9. Audit Events

New events in `wims.audit_log`:

Event	Entity	When
<code>`BLOCK_DEVICE_TOKEN`</code>	<code>`device_blocklist`</code>	Admin blocks a device (single or bulk)
<code>`UNBLOCK_DEVICE_TOKEN`</code>	<code>`device_blocklist`</code>	Admin unblocks a device

10. Celery Tasks

Task	Schedule	Purpose
<code>`tasks.device_blocklist.resync_device_blocklist`</code>	Every 5 min	Restore <code>`device:block:{hash}`</code> Redis keys from Postgres (same pattern as existing <code>`resync_ip_blocklist`</code>)
<code>`tasks.data_retention.purge_device_blocklist`</code>	Daily	Hard-delete expired device blocklist entries

10.1 Retention config

New config key: `retention.device_blocklist_days` (default: 365 days, same as IP blocklist).

11. Env Vars

Var	Purpose
<code>`DEVICE_TOKEN_SIGNING_KEY`</code>	HMAC-SHA256 signing key for device tokens (version 1)
<code>`DEVICE_TOKEN_SIGNING_KEY_V2`</code>	Rotated signing key (version 2, optional)
<code>`DEVICE_TOKEN_SIGNING_KEY_ACTIVE_VERSION`</code>	Active key version for new tokens (default: <code>`1`</code>)

Follows the existing `WIMS_MASTER_KEY` / `WIMS_KEY_CURRENT_VERSION` pattern.

12. Accepted Limitations

1. **Cookie deletion bypasses device block** — A sophisticated attacker can clear cookies and receive a new token. This is an accepted gap (abuse friction, not a hard security boundary). IP fallback + nginx/Suricata layers still catch uncooked traffic.
2. **Device token identifies browser installation, not physical device** — Same user on different browsers = different tokens. Same user clearing cookies = different token.
3. **Bot traffic without cookies** — Scripts that never accept cookies get no token. They fall through to IP blocking, nginx bot-blocker, and Suricata detection.
4. **Device hash in security logs is inferred, not packet-level** — The device token is correlated via Redis telemetry, not observed by Suricata. Admins can see `device_correlation_source` and judge reliability.

13. Out of Scope

- **Browser fingerprinting** — Deferred to a future iteration as a risk-scoring signal only, not a block key.
- **Registered device enrollment for BFP staff** — Documented in [system-wiki/decisions/0002-deferred-registered-device-enrollment.md](#). Deferred due to workflow friction.
- **nftables/iptables-level MAC blocking** — Not feasible from a remote VPS. Only applicable if on-premises deployment is added.
- **njs or Lua in nginx for header-based blocking** — The escalation is handled in the Python middleware layer.

14. Implementation Order

Phase	What	Dependencies
1	<code>`wims.device_blocklist`</code> table + <code>`services/device_blocklist.py`</code> + RLS	PostgreSQL migration
2	<code>`device_token_middleware`</code> + <code>`device_block_middleware`</code> + <code>`GET /api/device/token`</code>	Env vars configured

Phase	What	Dependencies
3	Redis telemetry correlation in Suricata ingestion + `security_threat_logs` columns	Phase 1 + 2
4	Device blocklist API endpoints + Celery resync task	Phase 1
5	Frontend: BlockedIpsPanel device tab + blocking flow	Phase 4
6	`check_device_abuse()` dependency + civilian route integration	Phase 2
7	Versioned keyring + rotation	Env vars configured